Doc. no.:	P0792R6
Date:	2022-01-17
Audience:	LEWG, LWG
Reply-to:	Vittorio Romeo <vittorio.romeo@outlook.com> Zhihao Yuan <zy@miator.net> Jarrad Waterloo <descender76@gmail.com>

`function_ref`: a type-erased callable reference

Table of contents

        - Theory
        - Test
        - On pointer-to-member
    - Wording
    - Feature test macro
    - Implementation Experience
    - Acknowledgments
    - References

Changelog

R6

- Avoid double-wrapping existing references to callables;
- Reworked the wording to follow the latest standardese;
- Applied changes requested by LWG (2020-07);
- Removed a deduction guide that is incompatible with explicit object parameters.

R5

- Removed “qualifiers” from `operator()` specification (typo);

R4

- Removed `constexpr` due to implementation concerns;
- Explicitly say that the type is trivially copyable;
- Added brief before synopsis;
- Reworded specification following P1369.

R3

- Constructing or assigning from `std::function` no longer has a precondition;
- `function_ref::operator()` is now unconditionally `const` -qualified.

R2

- Made copy constructor and assignment operator `= default` ;
- Added *exposition only* data members.

R1

- Removed empty state, comparisons with `nullptr`, and default constructor;
- Added support for `noexcept` and `const`-qualified function signatures;
- Added deduction guides similar to `std::function`;
- Added example implementation;
- Added feature test macro;
- Removed `noexcept` from constructor and assignment operator.

Abstract

This paper proposes the addition of `function_ref<R(Args...)>`, a *vocabulary type* with reference semantics for passing entities to call, to the standard library.

Design considerations

This paper went through LEWG at R5, with a number of consensus reached and applied to the wording:

1. Do not provide `target()` or `target_type`;
2. Do not provide `operator bool`, default constructor, or comparison with `nullptr`;
3. Provide `R(Args...) noexcept` specializations;
4. Provide `R(Args...) const` specializations;
5. Require the target entity to be *Lvalue-Callable*;
6. Make `operator()` unconditionally `const`;
7. Choose `function_ref` as the right name.

One design question remains not fully understood by many: how should a function, a function pointer, or a pointer-to-member initialize `function_ref`? The sixth revision of the paper aims to provide full background and dive deep into that question.

Question 1

There has been a concern on whether

```
int f();

function_ref fr = &f;
fr();
```

creates a `function_ref` object that points to a temporary function pointer and results in undefined behavior when called on the last line.

LEWG believes that `fr` should not be dangling here; it should store the function pointer. This is also `folly::FunctionRef` (https://github.com/facebook/folly/blob/main/folly/Function.h), `gdb::function_view` (https://github.com/bminor/binutils-gdb/blob/master/gdbsupport/function-view.h), and `type_safe::function_ref` (https://github.com/foonathan/type_safe/blob/main/include/type_safe/reference.hpp) (by Jonathan Müller)'s behavior.

P0792R5[1] believes that `fr` should be dangling here because the function pointer here is a prvalue callable object, and the nature of the problem is indifferent from `string_view` [2]. This is `llvm::function_ref` (https://github.com/llvm/llvm-project/blob/main/llvm/include/llvm/ADT/STLExtras.h), `tl::function_ref` (https://github.com/TartanLlama/function_ref/blob/master/include/tl/function_ref.hpp) (by Sy Brand), and P0792R5 sample implementation (https://github.com/vittorioromeo/Experiments/blob/master/function_ref.cpp)'s behavior.

LWG inferred and recognized the 2nd behavior from P0792R5 wording.

This paper (R6) supports the 1st behavior.

Question 2

The question above leads to the follow-up question: should `fr` be dangling if assigned a function?

```
int f();

function_ref fr = f;
fr();
```

LEWG believes that this case is indifferent to Question 1 and should work. This is `folly::FunctionRef` (https://github.com/facebook/folly/blob/main/folly/Function.h), `llvm::function_ref` (https://github.com/llvm/llvm-project/blob/main/llvm/include/llvm/ADT/STLExtras.h), `tl::function_ref` (https://github.com/TartanLlama/function_ref/blob/master/include/tl/function_ref.hpp), and P0792R5 sample implementation (https://github.com/vittorioromeo/Experiments/blob/master/function_ref.cpp)'s behavior assuming the C++ implementation supports converting a function pointer to an object pointer type or vice versa. This is also `gdb::function_view` (https://github.com/bminor/binutils-gdb/blob/master/gdbsupport/function-view.h) and `type_safe::function_ref` (https://github.com/foonathan/type_safe/blob/main/include/type_safe/reference.hpp)'s behavior because their implementation involves a union.[3]

P0792R5's design is that, `function_ref` references nothing but *callable objects* [func.def] (https://eel.is/c++draft/func.def#4). A function is not an object, so there is nothing we can reference. The function should decay into a function pointer prvalue, and we are back to Question 1. No known implementation supports this behavior.

LWG inferred and recognized the 1st behavior from P0792R5 wording.

This paper (R6) supports the 1st behavior.

Question 3

The last question is about pointer to members.

```
struct A { int mf(); } a;

function_ref fr = &A::mf;
fr(a);
```

Does the above have defined behavior?

LEWG made a poll and gave permission to "fully support the `Callable` concept at the potential cost of `sizeof(function_ref) >= sizeof(void(*)()) * 2`," 6/4/0/0/0 in 2018-6. This implies that an implementation should expand its union to store the pointer to member functions. No known implementation supports this behavior.

P0792R5 emphasizes that pointer-to-member objects are *callable*. Therefore we should point to those objects and deal with the potential dangling issue with tools. All the implementations mentioned above except `type_safe::function_ref` (https://github.com/foonathan/type_safe/blob/main/include/type_safe/reference.hpp) give this behavior.

`type_safe::function_ref` (https://github.com/foonathan/type_safe/blob/main/include/type_safe/reference.hpp) does not support initializing from a pointer-to-member and suggests the users to opt-in the 2nd behavior with `std::mem_fn(&A::mf)`.

LWG recognized the 2nd behavior while reviewing P0792R5 and generally agreed that a two-pointer `function_ref` is fat enough.

This is an open question in R6 of the paper, even though we bring in new information. We will discuss all options and their defenses.

Discussion

Is `std::function_ref` "parameter-only?"

Some people believe that `std::function_ref` only needs to support being used as a parameter.[4] In a typical scenario,

```
auto retry(function_ref<optional<payload>()> action);
```

there is no lifetime issue and all the three questions above do not matter:

```
auto result = retry(download); // always safe
```

However, even if the users use `function_ref` only as parameters initially, it's not uncommon to evolve the API by grouping parameters into structures, then structures become classes with constructors… And the end-users of the library will be tempted to prepare the parameter set before calling into the API:

```
retry_options opt(download, 1.5s);
auto result = retry(opt); // safe...?
```

If `download` is a function, the answer to Question 1 will now decide whether the code has well-defined behavior.

Is this only a lifetime issue?

The three questions we show above are often deemed "lifetime issues." Being a lifetime issue in C++ implies a misuse and a workaround. Moreover, users can look for lifetime issues with tools such as lifetime checkers, sanitizers, etc.

However, although we demonstrate the problem as a lifetime issue, the solutions we chose can inevitably impact the semantics of `function_ref`.

In the following line,

```
retry_options opt(p, 1.5s);
```

`p` is a pointer to function. It's an lvalue, so there is no lifetime issue. But if we go with the 2nd behavior when answering Question 2, a latter code that rebinds `p` to something else

```
p = winhttp_download;
```

will replace `opt`'s behavior.

One might argue that this behavior difference wouldn't be visible if `function_ref` were used only in parameter lists. However, this is not true either. If that function is a coroutine, the caller side can rebind `p` and ultimately change the behavior of the copy after the coroutine resumes.

Is this an issue or non-issue?

One might argue that what is stated so far are all corner cases. For example, lambda expressions create rvalue closures, so they always outlive `function_ref` immediately if you initialize `function_ref` variables with them. So what motivates us to guarantee more for functions and even function pointers?

The situation is that no implementation in the real world dangles in all three cases. Inevitably, some guarantees have become features in practice. For example,

```
struct retry_options
{
    retry_options(/* ... */) /* ... */ {}

    function_ref<sig> callback = fn_fail;
    seconds gap;
};
```

is a natural way to replace the default behavior with a specific one that reports failure. In addition, this style will become necessary after upgrading to `std::function_ref` as it has no default constructor.

In short, we're standardizing existing practice and can't leave "production" behavior implementation-defined. We expect the committee to give firm answers to these questions.

Rationale

Background knowledge

We have always treated `function_ref` as a `std::function` or `std::move_only_function` with reference semantics. But when applying this understanding, we worked out certain aspects of `function_ref` based on existing jargons, while these jargons reflect a contrived view of `std::move_only_function`.

> The `move_only_function` class template provides polymorphic wrappers that generalize the notion of a callable object. – [func.wrap.move.class]/1 (https://eel.is/c++draft/func.wrap.move.class#1)

> A *callable object* is an object of a callable type. – [func.def]/4 (https://eel.is/c++draft/func.def#4)

> A *callable type* is a function object type or a pointer to member. – [func.def]/3 (https://eel.is/c++draft/func.def#3)

In other words, from the library's point of view, function is not "callable." A function, reference to function, or reference to a callable object, can be used in `std::invoke`, and they satisfy the new `invocable` concept, but `std::move_only_function` is unaware of their existence. `move_only_function` stores callable objects (i.e., target objects). As long as something is to initialize that object, `move_only_function` is happy to accept it.

So defining `function_ref` as a "reference to a callable" leads to a problem: what do we do when there is no "callable object," invent one?

But `function_ref` had never meant to initialize any target object by treating the argument to `function_ref(F&&)` as an initializer. To `move_only_function`'s constructor, its argument is an initializer; if the argument is an object, the entity to call is always a different object – a distinct copy of the argument. To `function_ref`'s constructor, its argument **is** the entity to call.

I think this is a fundamental difference between `function_ref` and `move_only_function`. In the two `(F&&)` constructors, `F&&` in `move_only_function(F&&)` is to forward value, while in `function_ref(F&&)`, `F&&`'s role is to pass an identity so that we can call the entity that owns the identity via the reference. In my definition, `F&&` is a **callable reference**.

Theory

By saying something "models after a pointer," I mean the identity of such thing is irrelevant to the identity of the entity to reach after dereferencing. In C++, the "identity" here can be an address as nothing is relocatable in the abstract machine. By saying something "models after a reference," I mean it is similar to a pointer but lets you use that thing in place of the dereferenced entity.

When saying something is a "callable reference," I mean that thing models after a reference and is invocable. For example, an object pointer is not a callable reference, but a function pointer is. A reference to a function (e.g., `void (&)(int)`) is naturally a callable reference. `reference_wrapper<T>` is also a callable reference if `T` is invocable.

`function_ref<S>` is a callable reference as well. And I believe that when initializing `function_ref` from another callable reference, the `function_ref` object should bind to the entity behind the other callable reference. Such a callable reference needs to be type-passing since the conversion only can work as if we have both the identity and the type to the referenced entity.

Test

What should happen when a `function_ref<S>` object is initialized from or binds to another object of the same type?

Both are type-erased callable references. By saying something is type-erasing, we mean there exists a type C that can be used in place of A and B without forming a dependent expression. Type-passing means that substituting A with B changes the type of the expression. But when copy-constructing `function_ref`, the arguments are of the same type. So you can say that an expression to initialize `function_ref<S>` is dependent on `function_ref<S>`. The source type is equivalent to a type-passing callable reference, so we duplicate the internal states to make the two objects behave as if they are bound to the same entity.

> Why bring up the suggestion to unwrap `reference_wrapper` again?

Doing so fixes an issue: if initializing an object of `function_ref<R(Args...) const>` from `std::ref(obj)` given a modifiable `obj`, the `const` in the signature is not propagated to `obj`. It is the opposite behavior compared to the following:

```
struct A
{
    int operator()() const;  // 1
    int operator()();        // 2
} obj;

function_ref<int() const> fr = obj;
fr();  // calls 1
```

If `obj` were `const`, `std::ref(obj)` produces `reference_wrapper<T const>`, which calls #2. This tells us `std::ref` had never meant to “ensure” that an object is modifiable. By not creating `function_ref` on top of `reference_wrapper`, we prevent dangling, avoid object pointers that point to function pointers once more, and preserve `reference_wrapper`’s design logic.

On pointer-to-member

We are back to the open question. Pointer-to-members are hard to categorize. You could imagine that a pointer-to-member is some callable reference, but the entity it references does not exist in the language.

I think all of the following arguments make sense:

1. Because their referenced entities do not exist in the language, the code that initializes `function_ref` from a pointer-to-member should be ill-formed.
2. Their referenced entities do not exist⟹they are not callable references. Therefore they should be treated as callable objects, even though they might be prvalues.
3. Themselves should be treated as callable references even though they refer to hypothetical things. `function_ref` 's internal pointer may be an object pointer, a function pointer, and a pointer-to-member.

Meanwhile, there are other ways to use pointer-to-members in `function_ref`. `std::mem_fn(&A::mf)` is an option. Another option is `nontype<&A::mf>` from P2511[5]:

```
struct A { int mf(); } a;

function_ref<int(A&)> fr = nontype<&A::mf>;  // not dangling
fr(a);
```

Interestingly, this makes `function_ref` comparable to a GCC feature that converts PMF constants to function pointers[6], except that the underlying implementation involves a thunk.

Wording

The wording is relative to N4901.

Add the template to [functional.syn] (https://eel.is/c++draft/functional.syn), header `<functional>` synopsis:

> [...]

```
  // [func.wrap.move], move only wrapper
  template<class... S> class move_only_function;        // not defined
  template<class R, class... ArgTypes>
    class move_only_function<R(ArgTypes...) cv ref noexcept(noex)>; // see below

  // [func.wrap.ref], non-owning wrapper
  template<class S> class function_ref;                 // not defined
  template<class R, class... ArgTypes>
    class function_ref<R(ArgTypes...) cv noexcept(noex)>;           // see below
```

[...]

Create a new section “Non-owning wrapper”, `[func.wrap.ref]` with the following:

General

[func.wrap.ref.general]

The header provides partial specializations of `function_ref` for each combination of the possible replacements of the placeholders *cv* and *noex* where:

- *cv* is either `const` or empty.
- *noex* is either `true` or `false`.

An object of class `function_ref<S>` stores a pointer to thunk and a bound argument entity. A *thunk* is a function where a pointer to that function is a perfect forwarding call wrapper [func.def] (https://eel.is/c++draft/func.def). The bound argument is of an implementation-defined type to represent a pointer to object value, a pointer to function value, or a null pointer value.

`function_ref<S>` is a trivially copyable type [basic.types] (https://eel.is/c++draft/basic.types).

Within this subclause, `call_args` is an argument pack used in a function call expression [expr.call] (https://eel.is/c++draft/expr.call) of `*this`, and `val` is the value that the bound argument stores.

Class template `function_ref`

[func.wrap.ref.class]

```
namespace std
{
  template<class S> class function_ref;       // not defined

  template<class R, class... ArgTypes>
  class function_ref<R(ArgTypes...) cv noexcept(noex)>
  {
  public:
    // [func.wrap.ref.ctor], constructors and assignment operator
    template<class F> function_ref(F*) noexcept;
    template<class F> function_ref(F&&) noexcept;

    function_ref(const function_ref&) noexcept = default;
    function_ref& operator=(const function_ref&) noexcept = default;

    // [func.wrap.ref.inv], invocation
    R operator()(ArgsTypes...) const noexcept(noex);
  private:
    template<class... T>
      static constexpr bool is-invocable-using = see below;   // exposition only
  };

  // [func.wrap.ref.deduct], deduction guides
  template<class F>
    function_ref(F*) -> function_ref<F>;
}
```

Constructors and assignment operator

[func.wrap.ref.ctor]

```
template<class... T>
  static constexpr bool is-invocable-using = see below;
```

If *noex* is true, `is-invocable-using<T...>` is equal to:

```
is_nothrow_invocable_r_v<R, T..., ArgTypes...>
```

Otherwise, `is-invocable-using<T...>` is equal to:

```
is_invocable_r_v<R, T..., ArgTypes...>
```

```
template<class F> function_ref(F* f);
```

Constraints:

- `is_function_v<F>` is `true`, and
- *`is-invocable-using`*`<F>` is `true`.

Effects: Constructs a `function_ref` object with the following properties:

- Its bound argument stores the value `f`.
- Its target object points to a thunk with call pattern `invoke_r<R>(val, call_args...)`.

```
template<class F> function_ref(F&& f);
```

Let `U` be `remove_reference_t<F>`. If `remove_cv_t<U>` is `reference_wrapper<X>` for some `X`, let `T` be `X`; otherwise, `T` is `U`.

Constraints:

- `remove_cvref_t<F>` is not the same type as `function_ref`, and
- *`is-invocable-using`*`<`*`cv`*` T&>` is `true`.

Effects: Constructs a `function_ref` object with the following properties:

- Its bound argument stores the value `addressof(static_cast<T&>(f))`.
- Its target object points to a thunk with call pattern `invoke_r<R>(obj, call_args...)` where `obj` is an invented variable introduced in:

```
cv T& obj = *val;
```

```
function_ref(const function_ref& f) noexcept = default;
```

Effects: Constructs a `function_ref` object with a copy of `f`'s state entities.

Remarks: This constructor is trivial.

```
function_ref& operator=(const function_ref& f) noexcept = default;
```

Effects: Replaces the state entities of `*this` with the state entities of `f`.

Returns: `*this`.

Remarks: This assignment operator is trivial.

Invocation

[func.wrap.ref.inv]

```
R operator()(ArgsTypes... args) const noexcept(noex);
```

Let `g` be the target object and `p` be the bound argument entity of `*this`.

Preconditions: `p` stores a non-null pointer value.

Effects: Equivalent to `return g(p, std::forward<ArgTypes>(args)...);`

Deduction guides

[func.wrap.ref.deduct]

```
template<class F>
  function_ref(F*) -> function_ref<F>;
```

Constraints: `is_function_v<F>` is `true`.

Feature test macro

Insert the following to [version.syn] (https://eel.is/c++draft/version.syn), header `<version>` synopsis, after `__cpp_lib_move_only_function`:

```
#define __cpp_lib_function_ref 20XXXXL // also in <functional>
```

Implementation Experience

Here is a macro-free implementation that supports `const` and `noexcept` qualifiers: [Github] (https://github.com/zhihaoy/function_ref_nontype) [Godbolt] (https://godbolt.org/z/991avvPxx)

Sy Brand's `tl::function_ref` [Github] (https://github.com/TartanLlama/function_ref) is distributed in vcpkg ports. This implementation does not support qualified signatures.

Many facilities similar to `function_ref` exist and are widely used in large codebases. Here are some examples:

- `llvm::function_ref` (https://github.com/llvm/llvm-project/blob/main/llvm/include/llvm/ADT/STLExtras.h) from LLVM[7]
- `folly::FunctionRef` (https://github.com/facebook/folly/blob/main/folly/Function.h) from Meta
- `gdb::function_view` (https://github.com/bminor/binutils-gdb/blob/master/gdbsupport/function-view.h) from GNU

Acknowledgments

Thanks to **Agustín Bergé**, **Dietmar Kühl**, **Eric Niebler**, **Tim van Deurzen**, and **Alisdair Meredith** for providing very valuable feedback on earlier drafts of this proposal.

Thanks to **Jens Maurer** for encouraging participation.

References

1. *function_ref: a non-owning reference to a Callable* http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0792r5.html (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0792r5.html) ↩

2. *std::string_view accepting temporaries: good idea or horrible pitfall?* https://www.foonathan.net/2017/03/string_view-temporary/ (https://www.foonathan.net/2017/03/string_view-temporary/) ↩

3. *Implementing function_view is harder than you might think* http://foonathan.net/blog/2017/01/20/function-ref-implementation.html (http://foonathan.net/blog/2017/01/20/function-ref-implementation.html) ↩

4. *On function_ref and string_view* https://quuxplusone.github.io/blog/2019/05/10/function-ref-vs-string-view/ (https://quuxplusone.github.io/blog/2019/05/10/function-ref-vs-string-view/) ↩

5. *Beyond operator(): NTTP callables in type-erased call wrappers* http://wg21.link/p2511r0 (http://wg21.link/p2511r0) ↩

6. Extracting the Function Pointer from a Bound Pointer to Member Function. *Using the GNU Compiler Collection (GCC)* https://gcc.gnu.org/onlinedocs/gcc/Bound-member-functions.html (https://gcc.gnu.org/onlinedocs/gcc/Bound-member-functions.html) ↩

7. The function_ref class template. *LLVM Programmer's Manual* https://llvm.org/docs/ProgrammersManual.html#the-function-ref-class-template (https://llvm.org/docs/ProgrammersManual.html#the-function-ref-class-template) ↩